

# SIMATS ENGINEERING

## ASSESSMENT TOOL 3

**COURSE NAME:** Database Management Systems

**COURSE CODE:** CSA05

---

### COURSE OUTCOME COVERED

**CO5:** *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Mock Interview (Low)

Assessment Tool Weightage: 30%

---

Dr

| QUESTIONS |                                                                                                                               |               | Total Marks: 50 |
|-----------|-------------------------------------------------------------------------------------------------------------------------------|---------------|-----------------|
| Q.No      | Question                                                                                                                      | Bloom's Level | Marks           |
| 1         | Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'      | BL3 – Apply   | 5               |
| 2         | Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'    | BL4 – Analyze | 5               |
| 3         | Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.' | BL3 – Apply   | 5               |
| 4         | Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'                           | BL3 – Apply   | 5               |
| 5         | Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?' | BL4 – Analyze | 5               |
| 6         | Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'    | BL4 – Analyze | 5               |
| 7         | Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'                    | BL4 – Analyze | 5               |
| 8         | Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'                 | BL3 – Apply   | 5               |
| 9         | Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'     | BL4 – Analyze | 5               |

|    |                                                                                                                                                  |               |   |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---|
| 10 | Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?' | BL4 – Analyze | 5 |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---|

### Code of Conduct:

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.

### Signature of the student:

### RUBRICS FOR CALCULATION:

| Criteria              | Weightage | Level 4<br>(Exemplary)                                                            | Level 3<br>(Proficient)                          | Level 2<br>(Developing)                            | Level 1<br>(Beginning)                |
|-----------------------|-----------|-----------------------------------------------------------------------------------|--------------------------------------------------|----------------------------------------------------|---------------------------------------|
| Technical Knowledge   | 25        | Demonstrates strong and accurate subject knowledge with in-depth understanding    | Shows good knowledge with minor gaps             | Basic knowledge with noticeable errors             | Lacks fundamental technical knowledge |
| Problem Solving       | 25        | Solves problems logically and applies concepts effectively in real scenarios      | Applies concepts with moderate accuracy          | Limited problem-solving ability; requires guidance | Unable to solve or apply concepts     |
| Analytical Thinking   | 15        | Analyzes questions critically and provides well-structured, logical answers       | Provides reasonable analysis with some structure | Superficial or partially structured responses      | No analytical thinking demonstrated   |
| Communication         | 15        | Communicates clearly, confidently, and professionally with proper terminology     | Communicates well with minor hesitation          | Communication lacks clarity or confidence          | Unable to communicate effectively     |
| Confidence            | 15        | Displays high confidence, positive attitude, and professional behavior throughout | Shows good confidence with minor issues          | Low confidence; inconsistent behavior              | Lacks confidence and professionalism  |
| Response to Questions | 10        | Responds accurately, promptly, and elaborates with relevant examples              | Responds correctly but with limited depth        | Partial responses; needs prompting                 | Unable to respond appropriately       |



GPS Map Camera

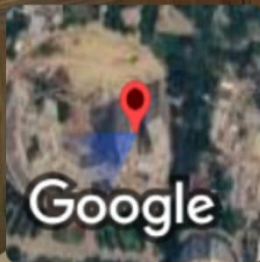
## Mevalurkuppam, Tamil Nadu, India



22g8+cwh, Mevalurkuppam, Tamil Nadu 602105, India

Lat 13.026414° Long 80.017009°

Thursday, 27/08/2026 01:57 PM GMT +05:30



## 1) Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'

Query processing in DBMS means converting a SQL query into an efficient execution plan and executing it.

There are mainly four steps. First is parsing, where the DBMS checks whether the query is syntactically and semantically correct. Second is translation, where the SQL query is converted into an internal form such as relational algebra. Third is query optimization, where the DBMS considers different ways to execute the query and selects the most efficient one. Finally, the selected plan is executed and the result is returned.

1. **Parsing:** Checks the SQL query for syntax and validity.
2. **Translation:** Converts the query into an internal representation, such as a relational algebra expression.
3. **Optimization:** Generates different execution plans and estimates their **cost** (I/O, CPU, memory).
4. **Execution:** Selects the lowest-cost plan and executes it to produce the result.

```
mysql> SELECT name FROM Employee  
-> WHERE department = 'IT';
```

The optimizer can either scan the entire Employee table or use an index on the department column. It estimates the cost of each option based on factors like **disk I/O, CPU usage, number of records, and available indexes**. It then chooses the plan with the **lowest estimated cost**.

---

## 2) Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'

“Heuristic optimization uses a set of predefined rules to improve a query. It does not calculate the actual cost of different execution plans. For example, it may perform selection before joins to reduce the number of rows processed.

Cost-based optimization, on the other hand, generates different execution plans and calculates their estimated cost based on factors like disk I/O, CPU, indexes, and number of records. It then chooses the plan with the lowest cost.

```
mysql> SELECT name  
-> FROM Employee  
-> WHERE department = 'IT';
```

A heuristic optimizer may directly apply the WHERE condition first. A cost-based optimizer may compare a **full table scan** with an **index scan** and choose whichever has lower estimated cost.

So, **heuristic optimization is faster and simpler**, while **cost-based optimization is more accurate but requires more computation**. Heuristics are useful for simple queries or when quick optimization is needed, while cost-based optimization is preferred for complex queries and large databases.”

### 3) Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'

“In a real project, I would first create the database and required tables. Then, in the application, I would use a **database driver** to establish a connection with the database. After connecting, I would execute SQL queries using objects provided by the connectivity API, process the results, and finally close the connection.

For example, in a Java application using MySQL, JDBC can be used:

```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/vehicle_rental",  
    "root",  
    ""  
);
```

Here, JDBC connects the Java application to the MySQL database.

The main difference is that JDBC is specifically designed for Java applications, whereas ODBC is a language-independent standard that allows applications to connect to different databases through ODBC drivers.

---

### 4) Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'

“ACID properties are the four properties that ensure database transactions are reliable and consistent. ACID stands for Atomicity, Consistency, Isolation, and Durability.

For example, consider a bank money transfer from Account A to Account B.

- **Atomicity:** The transaction should happen completely or not happen at all. If ₹1,000 is deducted from A but cannot be added to B, the transaction should be rolled back.
- **Consistency:** The transaction must keep the database valid. The money should be transferred without violating account balance rules.
- **Isolation:** If two transactions happen at the same time, they should not interfere with each other or produce incorrect balances.
- **Durability:** Once the transfer is successfully completed, the updated balances must remain saved even if the system crashes.

So, ACID properties ensure that transactions are accurate, reliable, and safe, especially in systems like banking, payment, and booking applications.”

**5) Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'**

“Concurrency control problems occur when multiple transactions access the same data at the same time.

- Dirty Read: A transaction reads data that another transaction has changed but not yet committed. If the second transaction rolls back, the first transaction has read incorrect data.
- Lost Update: Two transactions update the same data, and one update overwrites the other.
- Unrepeatable Read: A transaction reads the same data twice but gets different values because another transaction modified it between the two reads.

2PL, or Two-Phase Locking, solves these problems by controlling access using locks. It has two phases: growing phase, where a transaction obtains locks, and shrinking phase, where it releases locks.

For example, if Transaction T1 is updating an account balance, 2PL can prevent T2 from reading or modifying that balance until T1 completes.

So, 2PL ensures proper isolation between transactions and helps prevent dirty reads, lost updates, and unrepeatable reads.”

---

**6) Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'**

“Deadlock occurs when two or more transactions are waiting for each other to release resources, so none of them can continue.

For example, T1 locks Table A and waits for Table B, while T2 locks Table B and waits for Table A. Both transactions keep waiting, causing a deadlock.

To detect deadlock, the DBMS creates a Wait-For Graph. Transactions are represented as nodes, and an edge from T1 to T2 means T1 is waiting for T2. If the graph contains a cycle, a deadlock exists. The DBMS can then abort one transaction to break the cycle.

Two common prevention strategies are:

1. Wait-Die: An older transaction is allowed to wait, while a younger transaction requesting a locked resource is rolled back.
2. Wound-Wait: An older transaction can force a younger transaction to roll back, while a younger transaction waits for an older one.

So, deadlock detection finds the problem using a cycle in the Wait-For Graph, while prevention strategies try to avoid the deadlock before it occurs.”

## **7) Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'**

“Immediate Update means changes made by a transaction can be written to the database before the transaction commits. So, if the transaction fails, the DBMS may need to UNDO those changes. It generally uses both UNDO and REDO during recovery.

Deferred Update means changes are not written to the database until the transaction successfully commits. If the transaction fails before committing, there is usually no need to undo database changes. It mainly uses REDO to restore committed changes if needed.

For example, in a banking system, immediate update can be useful when changes need to be written quickly, while deferred update is preferred when we want simpler recovery and can delay writing changes until commit.

So, Immediate Update provides more flexibility but has more complex recovery, whereas Deferred Update is simpler but delays database updates until commit.”

---

## **8) Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'**

“Shadow Paging is a database recovery technique where the DBMS maintains two page tables: a shadow page table and a current page table.

The shadow page table points to the original database pages. When a transaction modifies data, the DBMS writes the changes to new pages instead of modifying the original pages. If the transaction commits, the current page table becomes the new valid version. If it fails, the DBMS simply uses the shadow page table to return to the original state.

For example, if a transaction changes a customer's balance, the original page remains unchanged, and the updated data is stored in a new page. If the transaction fails, the original page can be used.

Advantages: It provides quick recovery, does not require complex log files, and avoids UNDO/REDO operations.

Limitations: It can cause extra storage and page copying, may lead to fragmentation, and is less suitable for very large databases with many concurrent transactions.

Compared to log-based recovery, Shadow Paging is simpler and can recover quickly, but log-based recovery is generally more flexible and scalable for large, heavily concurrent database systems.”

## 9) Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'

“A serializable schedule is a schedule where multiple transactions execute concurrently, but the final result is the same as if those transactions had been executed one after another in some serial order.

For example, consider two transactions:

**T1: Read(A) → Write(A)**

**T2: Read(A) → Write(A)**

To check conflict serializability, we create a precedence graph. Each transaction is represented as a node. If two conflicting operations occur and one transaction comes before the other, we draw a directed edge.

For example:

**T1 → T2**

means T1 must execute before T2.

After creating the graph, we check for a cycle.

- No cycle → Conflict serializable
- Cycle exists → Not conflict serializable

So, the precedence graph helps us determine whether a concurrent schedule is equivalent to some serial execution.”

---

## 10) Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'

“Two-Phase Locking (2PL) is a concurrency control protocol that ensures transactions are executed in a way that maintains conflict serializability. It has two phases: a growing phase, where locks are acquired, and a shrinking phase, where locks are released.

There are three main variations:

- Basic 2PL: A transaction can acquire locks during the growing phase, but once it releases a lock, it cannot acquire any new locks.
- Strict 2PL: All exclusive locks are held until the transaction commits or aborts. This prevents other transactions from reading or modifying uncommitted data.
- Rigorous 2PL: Both shared and exclusive locks are held until commit or abort, providing even stronger control.

For example, if T1 updates an account balance, Strict 2PL keeps the exclusive lock until T1 commits, so T2 cannot access the uncommitted update.

Strict 2PL is commonly used because it provides good concurrency while preventing problems like dirty reads and cascading rollbacks, making recovery easier.

So, in simple terms: Basic 2PL ensures serializability, Strict 2PL also simplifies recovery, and Rigorous 2PL provides the strongest locking discipline.”